

Object



CONTENT

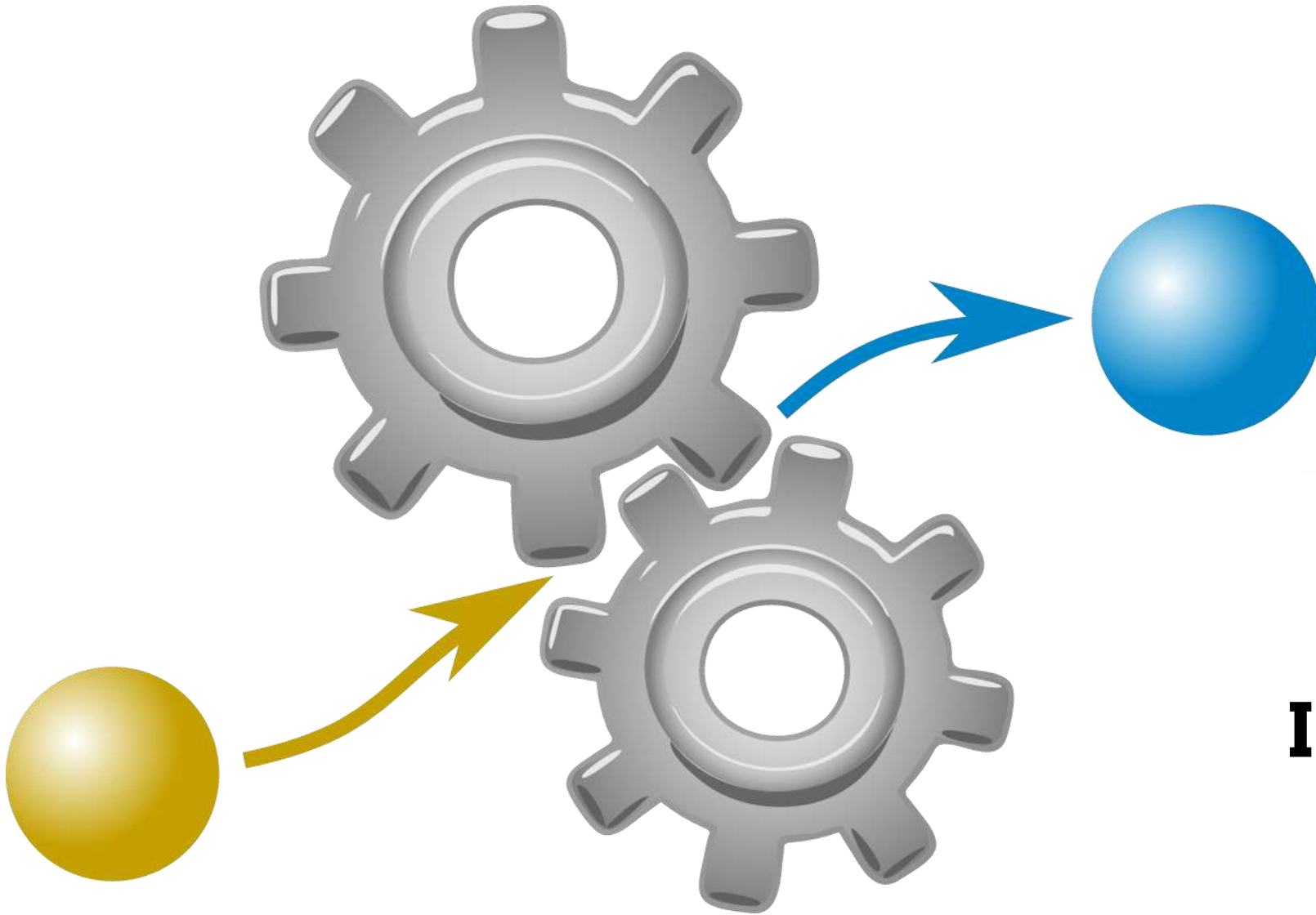
01 Introduction

02 Math Object

03 Number Methods

04 Date & Time





01

Introduction

Introduction

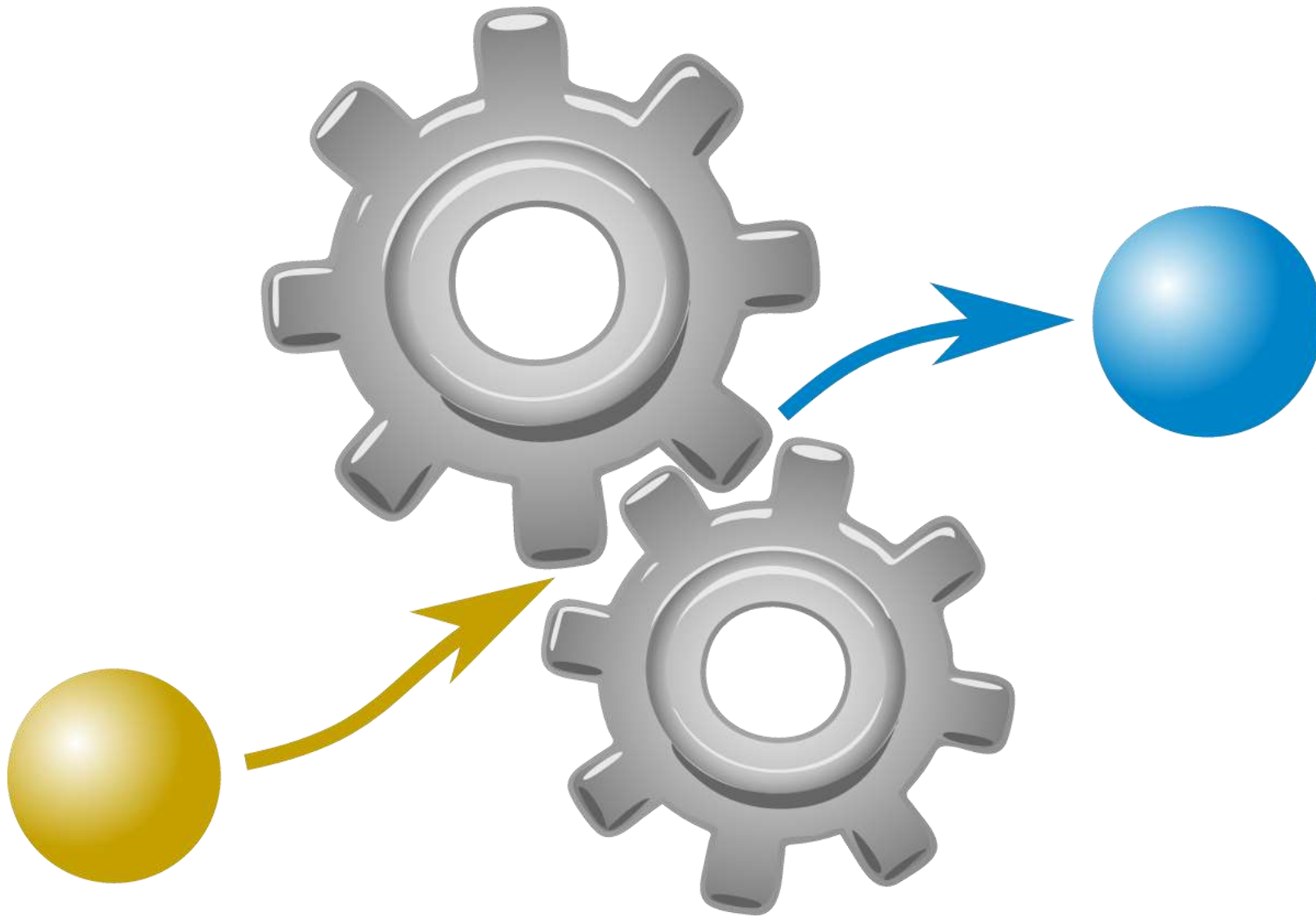
An object is a collection of key-value pairs, where:

- **Keys (properties) are always strings (or symbols),**
- **Values can be any data type — string, number, array, function, or even another object.**

Syntax:

```
const person = {  
  name: 'John',  
  age : 25,  
  isStudent: true,  
  greet: function () {  
    console.log(`My name is ${this.name} and i am ${this.age}yrs old.`);  
  }  
};  
person.greet()
```





02

Access

Accessing Object Properties

Dot Notation:

Dot notation is a way to access properties and methods of an object using a **dot** (.) followed by the property name.

Syntax :

objectName.propertyName

Syntax :

```
const person = {  
  name: "Nitish",  
  age: 25,  
  city: "Deoghar"  
};
```



Accessing Object Properties

Accessing a Property:

```
console.log(user.name); // John
```

Updating a Property:

```
user.age = 26;  
console.log(user.age); // 26
```

Adding a New Property:

```
user.country = "India";  
console.log(user.country); // "India"
```

Accessing a Method:

```
const car = {  
  brand: "Toyota",  
  start() {  
    console.log("Engine started");  
  }  
};  
  
car.start(); // "Engine started"
```



Limitations of Dot Notation:

Cannot Use Dynamic Property Names

```
const prop = "name";  
const user = { name: "Nitish" };  
  
console.log(user.prop); // ✗ undefined
```

Cannot Access Properties with Invalid Identifiers

Invalid identifiers include:

- Names with spaces
- Names with hyphens (-) or special characters
- Keys that start with a number



Limitations of Dot Notation:

```
const book = {  
  "book title": "JavaScript Guide",  
  "1stEdition": "Yes",  
  "author-name": "Nitish"  
};
```

```
console.log(book.book title);           // ✗ SyntaxError  
console.log(book.1stEdition);           // ✗ SyntaxError  
console.log(book.author-name);          // ✗ Error (evaluates as book.author - name)
```

No Support for Property Names with Special Symbols

```
const obj = {  
  "@id": 123  
};  
console.log(obj.@id);                  // ✗ SyntaxError
```



Limitations of Dot Notation:

No Expression Evaluation After Dot :

```
const user = { name: "Nitish" };  
const key = "name";  
  
console.log(user.key); // ✗ undefined
```

Note : Dot notation cannot compute or evaluate expressions after the dot — it takes it **literally**.

No Access to Symbol-keyed Properties

```
const sym = Symbol("id");  
const obj = { [sym]: 101 };  
  
console.log(obj.sym); // ✗ undefined
```

Note : Dot notation **only works for string-based keys**, not Symbols.



Limitations of Dot Notation:

No Expression Evaluation After Dot :

```
const user = { name: "Nitish" };  
const key = "name";  
  
console.log(user.key); // ✗ undefined
```

Note : Dot notation cannot compute or evaluate expressions after the dot — it takes it **literally**.

No Access to Symbol-keyed Properties

```
const sym = Symbol("id");  
const obj = { [sym]: 101 };  
  
console.log(obj.sym); // ✗ undefined
```

Note : Dot notation **only works for string-based keys**, not Symbols.



Bracket Notation

Bracket notation is a way to access, update, or define properties of a JavaScript object using square brackets [].

Unlike dot notation, bracket notation allows dynamic, computed, and special key access.

Syntax :

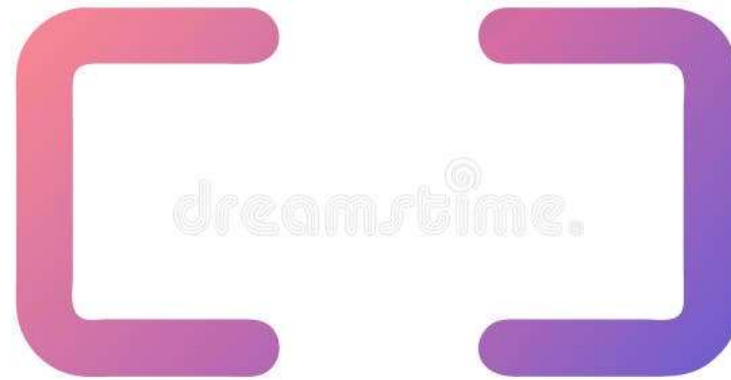
`object["propertyName"]`

Or

`object[variableThatStoresKey]`



Bracket Notation



Example :

```
const person = {  
  name: "John",  
  age: 25  
};
```

```
console.log(person["name"]);
```

```
// ✓ John
```



Bracket Notation

When to Use Bracket Notation :

Dynamic Property Access

```
const key = "age";  
console.log(person[key]); // ✔ 25
```

Note : Use bracket notation when the key is **stored in a variable**.

Keys with Special Characters / Spaces

```
const product = {  
  "product-name": "Phone",  
  "release date": 2025  
};
```

```
console.log(product["product-name"]); // ✔ Phone  
console.log(product["release date"]); // ✔ 2025
```

Note :

Cannot be accessed with dot notation:
product.product-name ✗



Bracket Notation

Numeric Keys or Symbol Keys

```
const obj = {  
  1: "One",  
  2: "Two"  
};
```

```
console.log(obj[1]); // ✓ One  
console.log(obj["1"]); // ✓ One — JS automatically converts string to number
```

Modify or Add Properties with Bracket Notation

```
person["city"] = "Deoghar"; // Add  
person["age"] = 26; // Update
```





Dot vs Bracket Notation

Feature / Use Case	✓ Dot Notation	✓ Bracket Notation
Basic key access (obj.name)	✓ Yes	✓ Yes
Key stored in a variable	✗ No	✓ Yes (obj[var])
Key has spaces ("first name")	✗ No	✓ Yes (obj["first name"])
Key has hyphens or special characters	✗ No	✓ Yes (obj["key-name"])
Numeric keys (obj[1] or "1")	⚠ Risky (converted to string)	✓ Yes
Symbol keys (Symbol("id"))	✗ Not supported	✓ Required (obj[sym])
Defining keys dynamically	✗ Not allowed	✓ Allowed ({ [key]: value })
Cleaner syntax	✓ Simpler and cleaner	✗ Slightly more verbose
When using computed or dynamic keys	✗ Fails	✓ Works



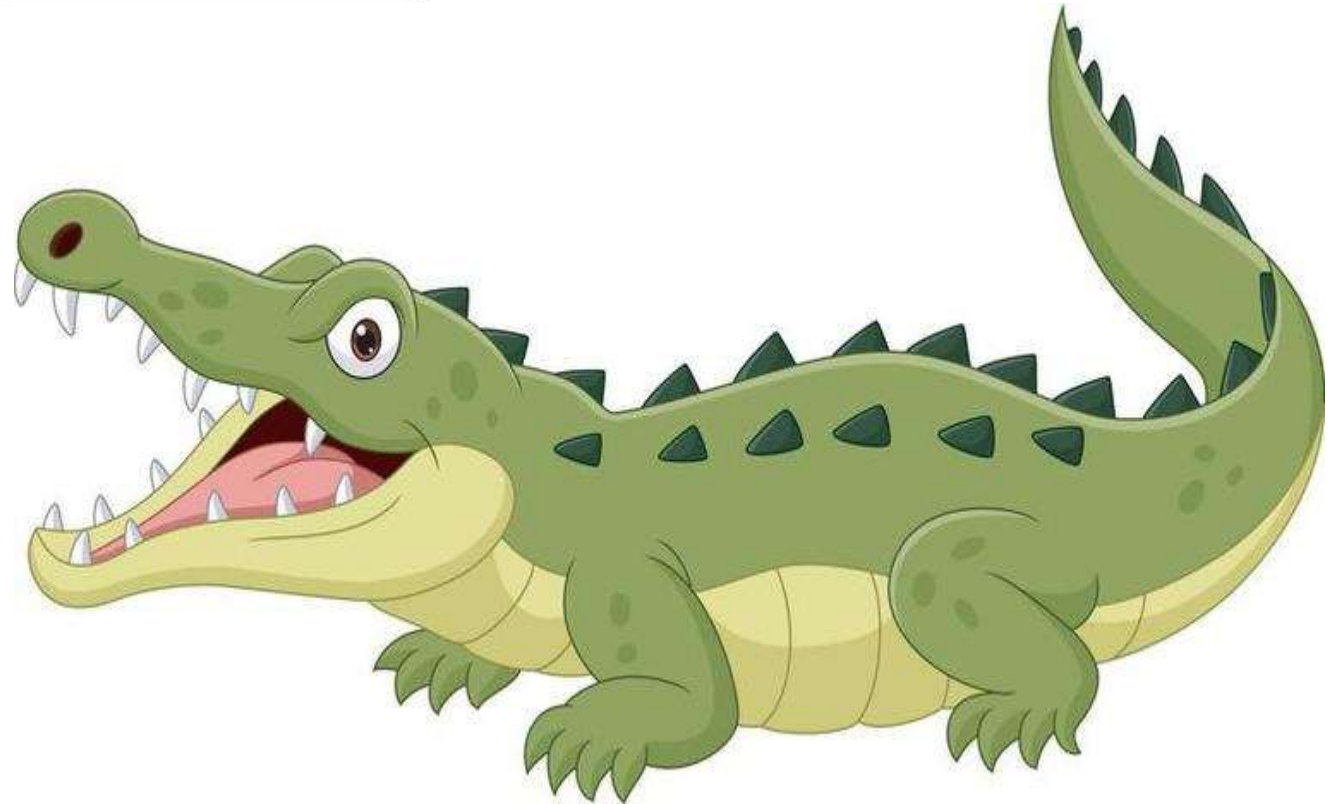
Modify

Adding, Updating, and Deleting

```
person.city = "Delhi";    // Add  
person.age = 26;          // Update  
delete person.isStudent;  // Delete
```

Checking Property Existence :

```
console.log("name" in person);    // true  
console.log(person.hasOwnProperty("age"));  // true
```



Iteration

for in Loop

Syntax :

```
for (let key in obj) {  
    //code  
}
```

Syntax :

```
for (let key in person) {  
  
    console.log(key + ": " + person[key]);  
  
}
```



Object Methods

Functions inside an object are called methods.

Example

```
const car = {  
  brand: "BMW",  
  start: function() {  
    console.log("Car Started!");  
  }  
};  
car.start();
```

this Keyword

```
const student = {  
  name: "Nitish",  
  showName: function() {  
    console.log(this.name);  
  }  
};  
student.showName();
```



Built-in Object Methods

Method	Use
Object.keys(obj)	Returns an array of keys
Object.values(obj)	Returns an array of values
Object.entries(obj)	Returns an array of [key, value] pairs
Object.assign(target, src)	Clones or merges objects
Object.freeze(obj)	Freezes object (immutable)
Object.seal(obj)	Prevents adding/deleting but allows modify



Nested Objects

Example

```
const user = {  
  name: "Nitish",  
  address: {  
    city: "Deoghar",  
    state: "Jharkhand"  
  }  
};  
console.log(user.address.city);    // Deoghar
```

Optional Chaining (?.) ✓ Modern JS

```
console.log(user.address?.city);    // safe access
```



Object Destructuring

- ❖ Destructuring is a syntax in JavaScript that allows you to unpack values from objects (or arrays) into distinct variables in a clean and readable way.

Example

```
const person = {  
  name: "Nitish",  
  age: 25,  
  city: "Deoghar"  
};
```

```
const { name, age } = person;  
console.log(name); // Nitish  
console.log(age); // 25
```

`{ name, age } = person` → extracts the name and age properties into variables.



Destructuring with Different Variable Names

➤ **`const person = { name: "Nitish", age: 25 };`**

```
const { name: fullName, age: personAge } = person;  
console.log(fullName); // Nitish  
console.log(personAge); // 25
```

You can rename keys while destructuring.

➤ **Default Values in Destructuring**

```
const person = { name: "Nitish" };
```

```
const { name, age = 18 } = person;  
console.log(age); // 18 (default value because age doesn't  
exist)
```



Nested Object Destructuring

Example

```
const student = {  
  name: "Aman",  
  address: {  
    city: "Ranchi",  
    state: "Jharkhand"  
  }  
};
```

```
const { address: { city, state } } = student;  
console.log(city); // Ranchi  
console.log(state); // Jharkhand
```

You can extract nested properties directly.



Destructuring in Function Parameters

Example

```
function showInfo({ name, age }) {  
  console.log(`Name: ${name}, Age: ${age}`);  
}
```

```
const person = { name: "Nitish", age: 25 };  
showInfo(person);
```

Very useful when you pass objects to functions.

➤ Destructuring with Rest Operator

```
const user = { id: 1, name: "Ram", age: 20 };
```

```
const { id, ...rest } = user;  
console.log(id); // 1  
console.log(rest); // { name: "Ram", age: 20 }
```

...rest collects remaining properties.



Spread Operator?

The spread operator (...) allows you to expand or spread out the elements of an object (or array) into another object or function.

It is part of **ES6 (ES2015)** and makes copying, merging, and adding properties easier.

Basic Syntax :

```
const newObject = { ...oldObject };
```

Feature	Benefits
Copy Objects	Easy cloning without mutation
Merge Objects	Combine multiple objects
Add/Update Properties	Create new object with additional data
Immutability	Avoid changing original object (Best Practice)



Spread Operator?

The spread operator (...) allows you to expand or spread out the elements of an object (or array) into another object or function.

It is part of **ES6 (ES2015)** and makes copying, merging, and adding properties easier.

Basic Syntax :

```
const newObject = { ...oldObject };
```

Feature	Benefits
Copy Objects	Easy cloning without mutation
Merge Objects	Combine multiple objects
Add/Update Properties	Create new object with additional data
Immutability	Avoid changing original object (Best Practice)



Spread Operator?

➤ Cloning an Object

```
const person = { name: "Nitish", age: 25 };  
const clone = { ...person };
```

```
console.log(clone); // { name: 'Nitish', age: 25 }
```

Creates a shallow copy (copies first level only).

➤ Adding New Properties

```
const student = { name: "Aman" };  
const newStudent = { ...student, course: "B.Tech" };
```

```
console.log(newStudent);  
// Output: { name: "Aman", course: "B.Tech" }
```



Spread Operator?

➤ Updating Existing Properties

```
const product = { name: "Laptop", price: 30000 };  
const updatedProduct = { ...product, price: 28000 }; // price updated  
  
console.log(updatedProduct);
```

➤ Merging Multiple Objects

```
const obj1 = { a: 1, b: 2 };  
const obj2 = { c: 3, d: 4 };  
const merged = { ...obj1, ...obj2 };  
  
console.log(merged); // { a: 1, b: 2, c: 3, d: 4 }
```

In case of same keys, the later object will override the previous one.



Shallow Copy

➤ What is Copying in JavaScript ?

When you **copy an object**, you either:

- **Shallow Copy** → copies only the first layer
- **Deep Copy** → copies everything, including nested objects.

➤ Shallow Copy

Shallow copy creates a **new object** but the **nested objects inside it are still referenced**.

Characteristics	Example
✓ Only top-level copied	Nested objects share reference
✓ Fast and simple	✗ Nested changes reflect in original



Shallow Copy

```
const user = {  
  name: "Nitish",  
  address: { city: "Deoghar" }  
};
```

// Shallow Copy

```
const copyUser = { ...user };
```

```
copyUser.address.city = "Delhi";
```

```
console.log(user.address.city);
```

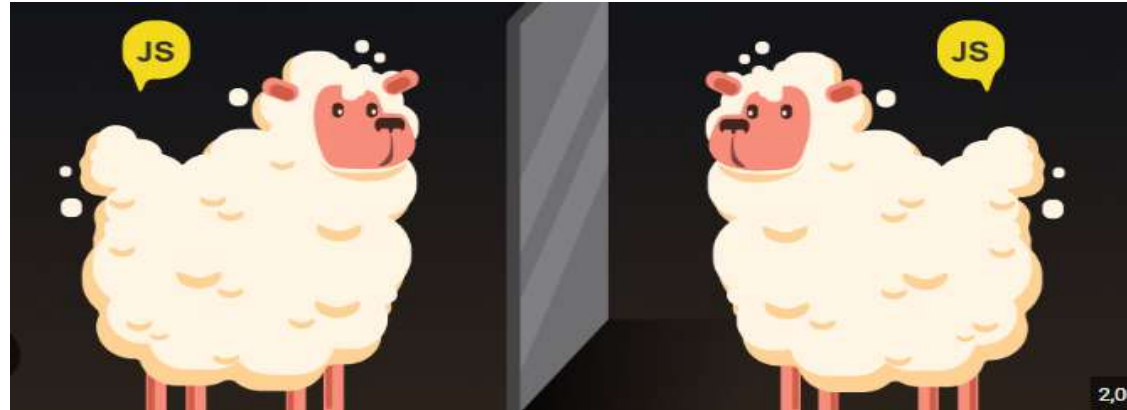
// Delhi 😬 (original modified!)

➤ Ways to Create Shallow Copy

- Spread Operator ({...obj})
- Object.assign({}, obj)
- Array methods (slice(), concat())



Deep Copy



Deep copy creates a completely independent clone, including all nested levels.

Characteristics	Example
✓ Fully independent copy	✗ Slightly slower
✓ Safe for complex objects	✓ Nested changes DO NOT affect original



Deep Copy

```
const user = {
  name: "Nitish",
  address: { city: "Deoghar" }
};
```

```
// Deep Copy using JSON method
const deepCopyUser = JSON.parse(JSON.stringify(user));
```

```
deepCopyUser.address.city = "Delhi";
```

```
console.log(user.address.city); // Deoghar 😊 (original unchanged)
```

➤ Ways to Create Deep Copy

Method	Notes
<code>JSON.parse(JSON.stringify(obj))</code>	✗ Loses functions, undefined, Date
Libraries like <code>lodash.cloneDeep()</code>	✓ Handles complex data properly
Manual recursion	✓ Complete control but complex



Shallow copy vs. Deep copy

Feature	Shallow Copy	Deep Copy
✓ Level of copy	Top-level only	Entire object with all levels
✓ Nested Object	Same reference (linked)	Fully new independent nested objects
✓ Common Methods	Spread, Object.assign()	JSON methods, lodash.cloneDeep()
✓ Speed	Faster	Slower for large data
✓ Use-Case	Simple objects	Nested/complex objects

